SOUP INTERNSHIP · TAKE-HOME TASK

AI Engineer

Work: 4–6 h · Deadline: 48 h · Hardware: free Colab T4 · Setup: pip install "soup-cli[train]"

MATERIALS

An open model up to 3B (for example Qwen2.5-1.5B-Instruct) and any open instruction dataset of 1–2k examples, in a language you can evaluate yourself. Your choice: justify it. Stack: Soup, or another one if you explain why.

TASK

Fine-tune the model with LoRA, merge the adapter into the base model, quantize it to 4-bit, and answer clearly: ship this artifact to users or not? A successful run, a falling loss and good answers on a couple of examples prove nothing.

1. Memory budget (before training). Estimate VRAM use for training and for export: weights, activations, logits, optimizer and adapter states, and the peak during merge and quantization. Show the calculation, compare it to the actual peak, and explain the gap.

2. Prove the model actually trained. Why doesn't a falling loss prove learning or an active adapter? Write a check that tells a truly trained model from a run that finished without meaningfully changing it, and that compares three artifacts: the base, the merged and the 4-bit model. Run it, include the output, and state what the check would NOT detect.

3. Silent failures. Find on your own what in the chain data → tokenizer and template → training → merge → quantization → inference can break without a single error. For each: what could go wrong, how you checked, whether Soup's built-in checks (data doctor, pre-flight, soup ship) catch it, and what evidence supports your conclusion. A run that finishes but is wrong is the failure we care about most.

4. Verdict. One call, SHIP or DON'T SHIP, backed by the memory analysis, logs, checks and the artifact comparison. If DON'T SHIP, state exactly what must change first.

DELIVERABLES

Report (≤2 pages) · raw timestamped logs · raw nvidia-smi output · the verification script from part 2 · the finished notebook or code · one paragraph on what surprised you and what still concerns you. Submit by email reply to team@trysoup.dev. Don't edit or hide failed results: a clear negative beats a quiet success. If something breaks, tell us what happened, what you tried, and what you learned.

AI TOOLS

Allowed (we use them too). Note where you used them, which suggestions you accepted, and what you verified, changed, or redid yourself. Not a penalty: we want to see how you work and where you spot an unreliable tool.

LIVE DISCUSSION

40 minutes, after submission. Don't pre-write answers; we want to think it through with you. Possible questions: The loss curve looks healthy: how did you prove training happened? What does your check miss? Do you trust the reported tok/s and VRAM? What would you investigate first with three days on rented H100s? What would you change in Soup so this failure can't pass soup ship again?

SCORING

Proving the model is real — 30% · silent failures — 20% · memory and hardware reasoning — 15% · live discussion — 35%. We're not just checking whether a metric improved. We want an engineer who questions assumptions, checks evidence, and understands what the system is actually doing.

Good luck. Be precise, honest, and curious.